

Writing `stress_area` from a Python mission

When the mission is written in Python (`local_planner / skytrack_autonomy`), **the framework does not produce the stressed-crop areas for you**. You have to write the detection result to one specific file. After the mission ends the desktop app collects that file and attaches it to the mission report at `status_summary.extras.stress_area`. If the file is missing, in the wrong place or in the wrong format, the report carries no `stress_area` and the detection part scores nothing.

A complete example ships next to this guide: `hackathon_example_v1.py` — survey the field, find the yellow crop, write `stress_area`, spray it, land.

Why Python code mode

The AI model in the UI only supports **detection models** (bounding boxes), not **segmentation**. A `stress_area` is defined by the outline (polygon) of the stressed patch, so if you want the real shape instead of a rectangle, use **Python code mode**: run your own model or algorithm, work out the stressed areas in ENU coordinates, and write the polygons yourself.

The path

```
/root/.ros/captures/stress_area.json
```

This is the only path that gets collected. A file written anywhere else is not attached.

- Create the parent directory yourself before writing (`parents=True, exist_ok=True`).
- There must be exactly **one** file named `stress_area.json` under `/root/.ros/captures`, subdirectories included. Do not leave backups or drafts with that name anywhere in there.
- Write **atomically**: write to a temp file under a different name (for example `.stress_area.json.tmp`), then `replace` it onto the real name. That way a mission stopped mid-write leaves the previous, still valid file instead of a truncated one.
- You may write several times during a mission, for instance after each area. Whatever is on disk when the mission ends is what gets collected.
- **Write before landing** — right after the survey stage, for example. Never after `land()`.

Format

```
{
  "stress_area": [
    {
      "class": "stressed",
      "polygon": [[x1, y1], [x2, y2], [x3, y3]]
    }
  ]
}
```

Field	Type	Meaning
stress_area	list	The areas you found; [] when there are none
class	string	Area label, use "stressed". Missing means "stressed"
polygon	list of [x, y]	The outline, in world ENU metres (x east, y north), at least 3 vertices

Also:

- The top level **must** be the object {"stress_area": [...]}. Do not write a bare list, and do not add other keys.
- The file must be valid JSON, UTF-8 encoded.
- A polygon does **not** need to repeat its first vertex at the end; the scorer closes the ring.
- Coordinates are world ENU, not NED and not image pixels. The pose is NED, so convert: east, north = pose.y, pose.x.
- Polygons with fewer than 3 vertices are dropped. Filter out tiny areas yourself to avoid noise.

Where this happens in the example

File: `hackathon_example_v1.py`

1. The path, declared once (line 38):

```
STRESS_AREA_PATH = Path("/root/.ros/captures/stress_area.json") # attached to the report by the app
```

2. The writer — atomic, and the shape of the file (lines 110–119):

```
def save_stress_areas(areas: list[dict[str, Any]]) -> None:
    STRESS_AREA_PATH.parent.mkdir(parents=True, exist_ok=True)
    tmp = STRESS_AREA_PATH.with_name(".stress_area.json.tmp")
    tmp.write_text(json.dumps({"stress_area": areas}, separators=(",", ":")))
    tmp.replace(STRESS_AREA_PATH)
```

3. Called after the survey, **before landing** (lines 82–83):

```
areas = stress_map.stress_areas()
save_stress_areas(areas)
```

4. Where each entry comes from (lines 177–189). `StressMap.stress_areas()` returns [{"class": "stressed", "polygon": [[x, y], ...], ...}] with x, y already converted to ENU metres.

Replace `StressMap` with your own detection, but keep the path, the format and the way the file is written.

Source and example code: `contests/skytrack-hackathon-2026/` in github.com/GetSkyTrack/skytrack-autonomy-example